LIBRARY MANAGEMENT SYSTEM

Technology used : Python 3.12| CSV|Logging|Pytest

Tools: VS Code, GitHub

Date: 16 october 2026

Team members:

Thamzid Hussain Shaik Gaffar: modules, packages, and dependency setup.

Bhuvaneswari Galam: exceptions, file handling, and validation.

Thupakula Divya: pytest coverage, integration, and final demo.

Contents:

1. Introduction
2. Project Objectives
3. Technologies Used
4. Project Structure
5. Concepts Covered
6. Project Implementation
7. Testing and Results
8. How to Run the Project
9. Limitations and Future Enhancements
10. Conclusion

1.Introduction:

The Library Management System is a Python-based mini project developed to manage basic book records. The application allows a user to add books, display available books, and store book details in a CSV file. It uses separate Python modules to keep the code organized and includes input validation, logging, exception handling, and automated tests.

This project provides practical experience in Python programming and demonstrates how individual components can be combined into a working application.

2. PROJECT OBJECTIVES:

To create a simple menu-driven Library Management System using Python.

To organize code into reusable modules and classes.

To validate book details before saving them.

To store and load book records using a CSV file.

To prevent duplicate book IDs.

To record important application events using logging.

To test key application behavior using Pytest.

To integrate the components and prepare a final demonstration.

3.TECHNOLOGIES USED :

Technology	**Purpose**
Python 3.12	Main programming language
Visual studio code	Code editor used to develop and run the project
CSV	Stores book records in a tabular text file
Logging	Records application events, warnings, and errors
Pytest	Runs automated tests
Git and GitHub	Can be used for version control and project sharing

4.PROJECT STRUCTURE:

The main application files are organized as follows:

Day5_Mini_Project/

book.py

library.py

main.py

test_library.py

books.csv

library.log

book.py: Defines the Book class and formats book information.

library.py: Manages the book collection, validates details, reads and writes CSV records, and logs events.

main.py: Displays the menu and connects user choices to library operations.

test_library.py: Contains automated tests for important behaviors.

books.csv: Stores book IDs, titles, and authors.

library.log: Stores logged application events and errors.

5. CONCEPTS COVERED:

5.1 Modules and Imports

A module is a Python file containing reusable code. The project imports the Book class from book.py into library.py. This separation makes the code easier to understand and maintain.

5.2 Classes and Objects

A class is a blueprint for creating objects. The Book class represents a book using its ID, title, and author. The Library class manages a collection of Book objects.

5.3 Exception Handling

Exception handling helps an application respond to errors in a controlled way. The project handles common file and CSV errors so that loading or saving problems can be reported.

5.4 File Handling

The application uses Python file operations to read and write book records. The with statement is used to manage files safely, and CSV data is stored in books.csv.

5.5 Input Validation

The application checks that book ID, title, and author are not empty. It also checks for duplicate book IDs before adding a new record.

5.6 Logging

Logging records important application events, warnings, and errors in library.log. This helps with troubleshooting and understanding application behavior.

5.7 Automated Testing

Pytest is used to execute test functions and check expected results with assertions. Automated tests help verify that important features continue to work after code changes.

6. PROJECT IMPLEMENTATION:

6.1 Add Book

The user selects the Add Book menu option and enters a book ID, title, and author. The application validates the fields, checks for duplicate IDs, creates a Book object, and saves the record to the CSV file.

6.2 Display Books

The Display Books option shows the books currently available in the library. If there are no records, the application displays a message indicating that no books are available.

6.3 CSV Persistence

Book records are stored in books.csv. When the application starts, it attempts to load existing records so that saved books remain available between runs.

6.4 Validation and Duplicate Prevention

Empty book fields are rejected. If a book ID already exists, the application returns an error message rather than adding another book with the same ID.

6.5 Logging and Error Handling

The application records events such as adding books and invalid choices. File-related errors are logged and reported to help the user understand when an operation fails.

7. TESTING AND RESULTS:

The test suite was executed from the project directory using the following command:

pytest -v

The latest recorded test run completed with 5 passed tests.

Test Case	Results
Book display information	PASSED
Add book successfully	PASSED
Display books	PASSED
Empty library message	PASSED
Duplicate book ID Validation	PASSED

These results confirm that the five listed behaviors passed in the recorded run. They do not imply that every possible input or error scenario has been tested.

8.HOW TO RUN THE PROJECT:

Open a terminal in the Day5_Mini_Project directory.

Run the application with:

python main.py

Run the automated tests with:

pytest -v

If Pytest is not installed in the active environment, install it using python -m pip install pytest. Use the project's actual dependency instructions if a requirements file is provided.

9. LIMITATIONS AND FUTURE ENHANCEMENTS

The current version focuses on adding and displaying books. Possible future improvements include:

Search for books by ID, title, or author.

Update and delete existing book records.

Add book issue and return functionality.

Track borrowers and due dates.

Add a graphical user interface.

Expand automated tests for file errors, invalid CSV headers, and other edge cases.

10.COMPLETION STATUS:

Task	Status
Python modules and classes	Completed
Library book management	Completed
CSV file handling and data persistence	Completed
Input validation and duplicate ID prevention	Completed
Exception handling	Completed
Logging	Completed
Pytest test cases	Completed
Project integration	Completed
Documentation	Completed
Final Demonstration	Completed

11. CONCLUSION:

The Library Management System demonstrates how Python can be used to build a small, structured application. It combines modules, object-oriented programming, CSV file operations, input validation, logging, and automated testing. The latest recorded test run passed all five implemented test cases. My assigned contribution focused on Pytest coverage, integration, and preparation for the final demonstration.

Overall status : Completed successfully